

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
INSTITUTO METRÓPOLE DIGITAL
PROGRAMA DE PÓS-GRADUAÇÃO EM TECNOLOGIA DA INFORMAÇÃO
Doutorado Profissional

Desenvolvimento de Software com IA

Atividade Prática Assíncrona

Configuração do Ambiente e Fluxo de Trabalho com IA

Aluno: Geomerez Raduan de Oliveira Bandeira

Programa: PPgTI, Doutorado Profissional

Professor: Prof. Dr. Jean Mário Moreira de Lima

Natal/RN
Agosto de 2026

Etapa 1. Configuração do Ambiente

Ferramenta escolhida. Eu usei o Cursor, versão **3.17.8** (versão estável, baseada no VS Code 1.128.0). Escolhi essa ferramenta porque já a uso no dia a dia e tenho licença paga. Ela atende o que a atividade pede na parte de IDE com assistente.

Pasta do laboratório. Criei a pasta `ia-dev-lab/`, como pede o enunciado. O arquivo de teste ficou em `ia-dev-lab/hello.py`.

Repositório Git/GitHub. Iniciei o repositório neste projeto e liguei ao GitHub:

`https://github.com/raduanoliveira/desenvolvimento_com_ia_aula_assincrona.git`

O primeiro commit (`ca1d094`, com a mensagem `first commit`) tem só o `README.md`. Depois publiquei a branch `main` com `git push -u origin main`.

Prompt digitado.

Crie um arquivo `hello.py` que imprima uma saudação e explique o que você fez.

Resposta da LLM. A LLM criou o arquivo `ia-dev-lab/hello.py` e deu esta explicação:

Criei o arquivo `hello.py` na pasta `ia-dev-lab`. O programa tem uma função `main` que imprime a frase “Olá! Este é o `ia-dev-lab` no Cursor.” Se o arquivo for executado direto, essa função é chamada e a saudação aparece no terminal.

Validação da execução. Rodei o `hello.py` de dentro da pasta `ia-dev-lab` e a saída foi: *Olá! Este é o `ia-dev-lab` no Cursor.*

Etapa 2. Contexto do Projeto

Para esta etapa defini um projeto de software: uma ToDo list em monorepo, com API em Laravel no backend e React com TypeScript, Context API e Material UI no frontend. O layout deve ser moderno. O código segue TDD, SOLID, camadas POSA e padrões GoF quando foi necessário. Implementei a camada Service no backend, que não vem por padrão em projetos Laravel: a regra de negócio fica no Service, não no controller. O ambiente é Docker. As pastas são `ia-dev-lab/backend/` e `ia-dev-lab/frontend/`.

O que é o AGENTS.md. Criei o arquivo `ia-dev-lab/AGENTS.md` no formato do slide da aula. Ele é o contexto permanente da IA neste projeto. As seções são:

- **Sobre o projeto:** diz o que o sistema faz (ToDo list, Laravel, React, MUI, Docker).
- **Comandos:** lista o que a IA deve rodar. Subir tudo, parar tudo, testes de unidade e de feature no backend, testes do frontend e migrations.
- **Convenções de código:** ideias gerais para a IA seguir: TDD, Service, Repository, SOLID, camadas POSA, GoF quando foi necessário, layout moderno e os tipos de teste. Sem citar arquivo do projeto.
- **Não fazer:** não pular camada, não pular teste, não trocar o MUI por CSS próprio, não instalar PHP ou Node na máquina.

Também criei uma regra customizada para o backend e outra para o frontend, na pasta de regras do Cursor dentro de `ia-dev-lab`.

Para conferir se a IA usa esse arquivo, pedi prompts que acionam os comandos do `AGENTS.md`.

Prompt digitado (subir o ambiente).

Suba o ambiente inteiro deste projeto.

Resultado da execução. Deu certo. O sistema subiu. Abri a lista de tarefas no navegador e ela estava vazia, o que era esperado. A tela do aplicativo também abriu sem erro.

Prompt digitado (testes).

Rode as checagens automáticas da API e da tela.

Resultado da execução. Deu certo. As checagens automáticas passaram: seis no servidor e cinco na tela do aplicativo.

Prompt digitado (parar o ambiente).

Pare o ambiente inteiro.

Resultado da execução. Deu certo. O sistema desligou. A lista de tarefas e a tela do aplicativo deixaram de abrir no navegador.

Etapa 3. Organização do Projeto

Nesta etapa organizei o laboratório por lado do sistema. A API ficou em **backend/**, dentro de **ia-dev-lab**. A tela ficou em **frontend/**. Evitei juntar tudo só por tipo de arquivo.

Criei o **README.md** na pasta **ia-dev-lab**, com visão geral, como subir o ambiente e os comandos principais. Também criei a pasta **docs/adr/** com quatro ADRs, que registram as escolhas deste projeto:

- **0001.** Cursor, porque eu já uso no dia a dia e tenho licença paga.
- **0002.** ToDo em monorepo com Laravel e React, porque é a stack que eu já conheço e consigo avaliar o que a IA gera.
- **0003.** Docker, para não instalar nada extra no Windows e para a IA subir e parar tudo pelos mesmos comandos.
- **0004.** Camada Service no Laravel, para o controller só tratar HTTP e para exercitar SOLID e camadas POSA.

A estrutura final se aproxima do modelo da aula: **AGENTS.md**, **README.md**, **docs/adr/** e o código separado por domínio.

Etapa 4. Boas Práticas de Prompt

A funcionalidade que eu escolhi foi arquivar uma tarefa, sem apagar o registro. Eu rodei o mesmo pedido de duas formas, em dois modelos, e conferi o código gerado. O detalhe também está em docs/prompts-comparacao.md. O código que ficou no laboratório é o do prompt eficaz no Grok 4.6.

Prompt fraco.

Adicione arquivar tarefa.

Prompt eficaz.

Contexto: este é um ToDo em monorepo. A API é Laravel. A tela é React com TypeScript, Context API e MUI. Arquivar tarefa é um caso de uso novo: a tarefa deixa a lista ativa sem ser apagada. O código já tem criar, listar, concluir e excluir.

Exemplo de padrão a seguir: teste primeiro. Um caso de uso por Service. Controller só HTTP. Persistência por interface de repositório. Na tela, o componente não chama a API direto.

Restrições: não colocar regra de negócio no controller nem na tela. Não pular camada. Não inchar um Service que já existe. Não pular testes.

Validação: teste da rota de arquivar e teste de unidade do Service. Na tela, ação de arquivar com teste de componente.

Modelos. Principal: Cursor Grok 4.6. Menor performance: Composer 2.5 Fast.

Grok 4.6, prompt fraco. A IA criou a rota de arquivar e um botão na lista. A regra ficou no controller: busca a tarefa, marca como arquivada e salva ali mesmo. Não criou Service novo. Não escreveu teste de arquivar. A listagem da API continua devolvendo tudo, inclusive o que foi arquivado. Na tela, o item arquivado permanece na lista, só muda o rótulo.

Grok 4.6, prompt eficaz. A IA criou um Service só para arquivar. O controller só chama esse Service. A lista ativa filtra o que está arquivado. Escreveu teste de unidade do Service, teste da rota e testes na tela. Eu conferi a execução: nove testes da API passaram e sete da tela passaram.

Composer 2.5 Fast, prompt fraco. Mesmo pedido curto. Desta vez a IA criou um Service de arquivar e o controller só delegou. Ainda assim não escreveu nenhum teste novo. A listagem escondeu arquivadas direto no repositório.

Composer 2.5 Fast, prompt eficaz. A IA também criou o Service, a rota e o botão. Escreveu teste de unidade, teste da rota e teste do botão na lista. Rodou os testes no Docker: oito da API passaram e seis da tela passaram. Não chegou no mesmo conjunto de testes do Grok com o prompt eficaz: faltou o teste de unidade da lista ativa e os testes do acesso à API e do estado da tela.

Diferenças que eu observei. No prompt fraco, o pedido curto não disse onde a regra deveria viver nem como validar. No Grok isso caiu no controller, sem teste e sem

tirar a tarefa da lista ativa. No Composer o código copiou o jeito dos outros Services, mas mesmo assim pulou os testes. No prompt eficaz, os dois modelos abriram um caso de uso novo, mantiveram o controller magro e escreveram testes. A diferença de qualidade apareceu mais no Grok, que cobriu unidade, rota e tela. O modelo mais fraco, com o prompt curto, não desandou tanto quanto o Grok no controller, porque o projeto já tinha Services para copiar. Ainda assim o prompt eficaz foi o que fez os dois modelos testarem de verdade.

Etapa 5. Integração com Git/GitHub e MCP

O repositório no GitHub já existia:

`https://github.com/raduanoliveira/desenvolvimento\_com\_ia\_aula\_assincrona.git`

Eu criei a branch `feature/setup-inicial` para as alterações do laboratório. Pedi à IA uma mensagem de commit a partir do diff. A mensagem que eu aceitei foi: *Monta o laboratório ToDo, o contexto da IA e o relatório da atividade.*

Abri um Pull Request dessa branch para a `main`, só para praticar o fluxo, sem fazer merge.

Servidor MCP. Eu criei o arquivo `.mcp.json` na raiz do repositório, com um servidor filesystem, como o enunciado pede. Há uma cópia no formato do Cursor. O comando do servidor é o `npx` do pacote de filesystem.

Eu tentei ligar a ferramenta a esse servidor e rodar um prompt que depende da conexão, no estilo: liste os arquivos alterados no último commit. A conexão não subiu. O `npx` e o Node não existem nesta máquina, porque eu escolhi não instalá-los no Windows e deixar tudo no Docker. Sem o `npx`, o Cursor não consegue iniciar o servidor MCP. Travei nesse ponto. O arquivo de configuração ficou no repositório, e este parágrafo registra a tentativa.